Universidade Federal do Piauí
Campus Senador Helvídio Nunes de Barros

Avaliação Teórica e Experimental dos Algoritmos de Ordenação MergeSort e TimSort

Discentes: João Vitor Santana Lima; Samuel Torres Vieira da Silva

Docente: Raí Araújo de Miranda
Disciplina: Projeto e Análise de Algoritmos

Sumário

1. Introdução
2. Demonstração
3. Resultados
4. Referências

Introdução

- A ordenação de dados é uma operação fundamental em Ciência da Computação;
- A eficiência dos algoritmos de ordenação impacta diretamente o desempenho dos sistemas, especialmente com grande volume de dados;
- Diferentes algoritmos apresentam comportamentos distintos conforme uma série de fatores.

Introdução

- Objetivos:
  - Realizar uma avaliação comparativa entre o *Merge Sort* e *Tim Sort*;
  - Relacionar resultados empíricos com previsões teóricas;
  - Analisar desempenho, complexidade assintótica e sensibilidade à ordenação inicial.

MergeSort

- O MergeSort é um algoritmo de ordenação baseado no paradigma “dividir e conquistar”;
- Inventado por John von Neumann em 1945;
- Muito usado em grandes volumes de dados.

Passos do MergeSort

O algoritmo funciona em três etapas :

1. Dividir;
2. Conquistar (resolver);
3. Combinar (merge).

Demonstração - MergeSort

Vetor de exemplo utilizado:

4	6	1	7	0	5	2	3

Demonstração - MergeSort

1* Passo - Dividir o vetor recursivamente ao meio

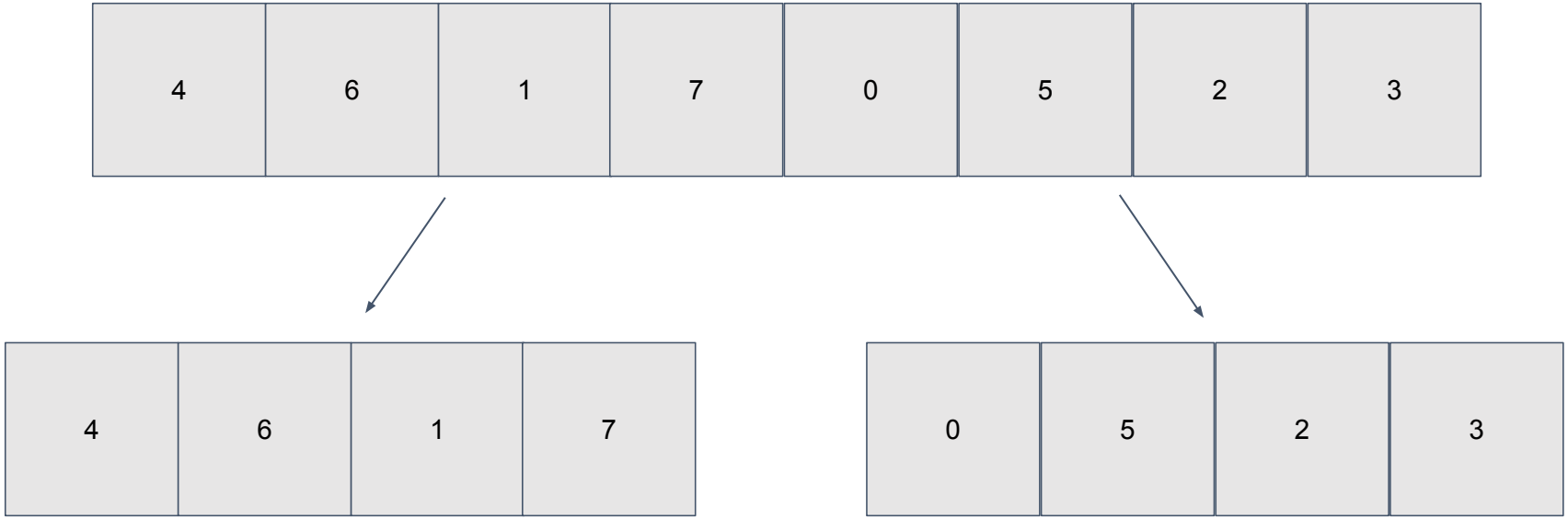
4	6	1	7	0	5	2	3

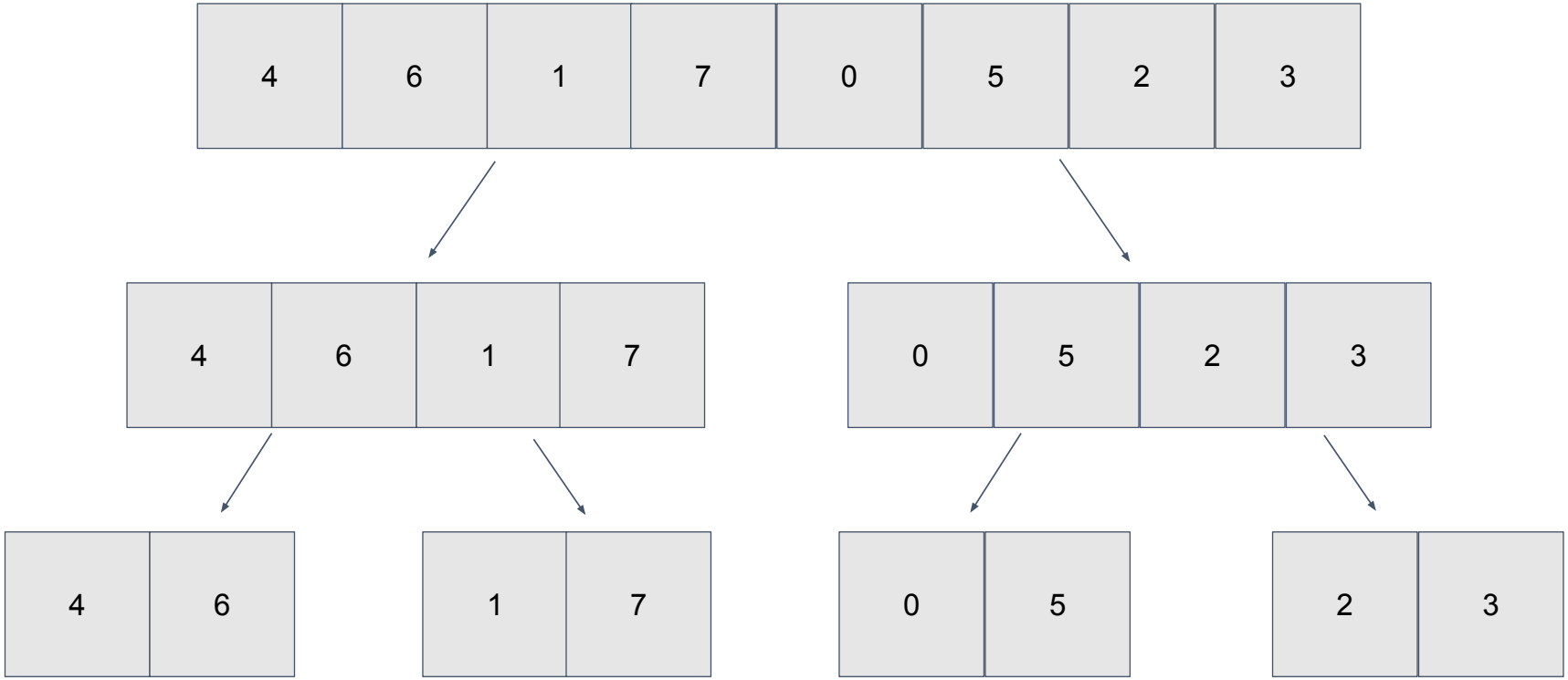
Demonstração - MergeSort

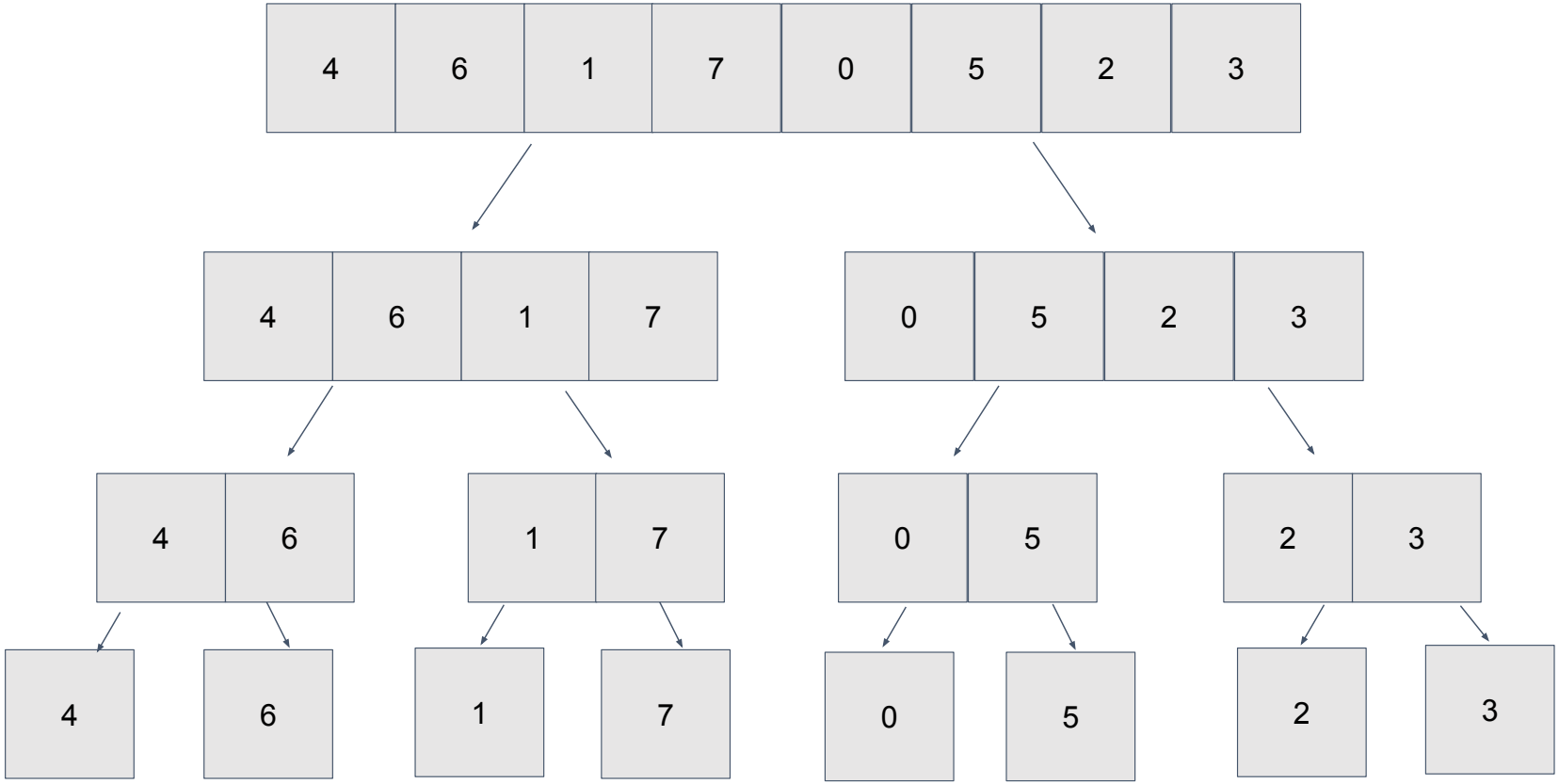
Tamanho do vetor = 8

4	6	1	7	0	5	2	3

Demonstração - MergeSort

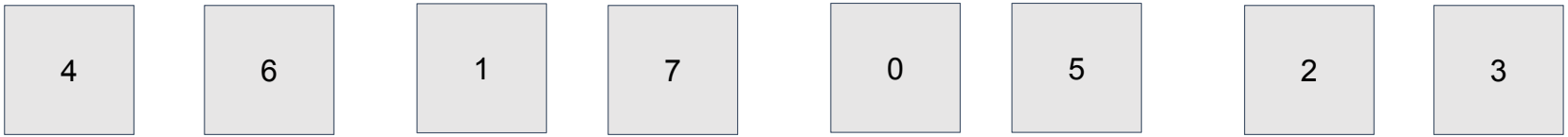



4 6 1 7 0 5 2 3
4 6 1 7
0 5 2 3
4 6
1 7
0 5
2 3


4 6 1 7 0 5 2 3
4 6 1 7
0 5 2 3
4 6
1 7
0 5
2 3
4
6
1
7
0
5
2
3

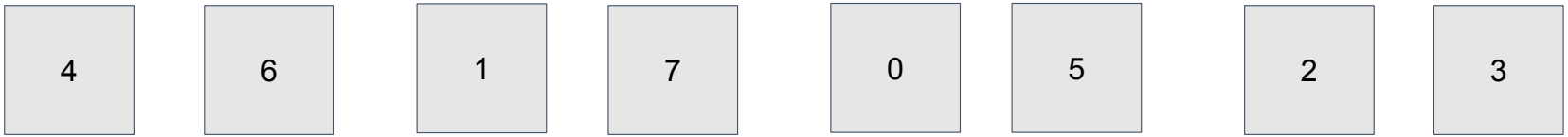
Demonstração - MergeSort

Cada parte tem apenas 1 elemento



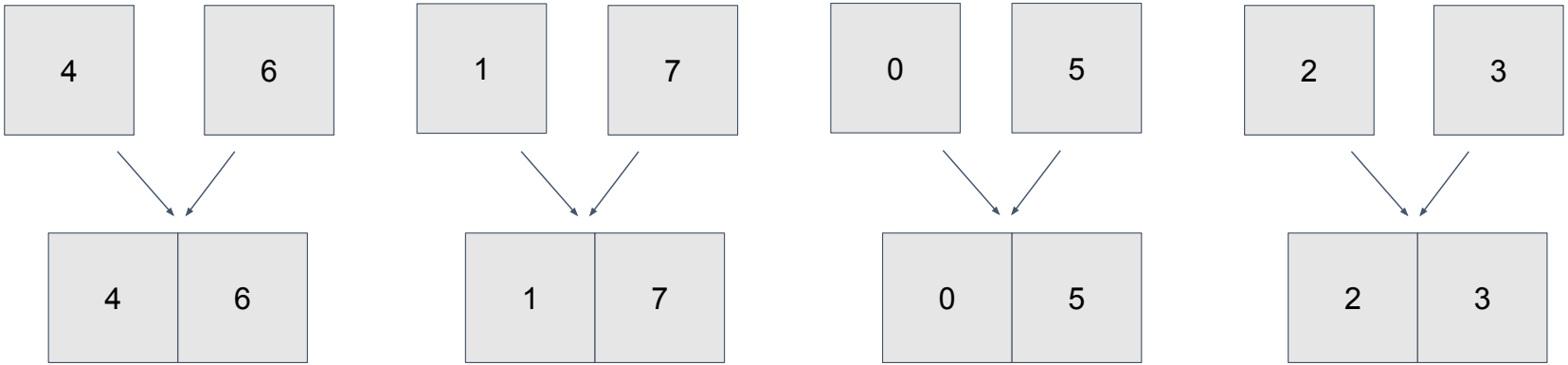
Demonstração - MergeSort

2* Passo - Fazer o merge ordenando

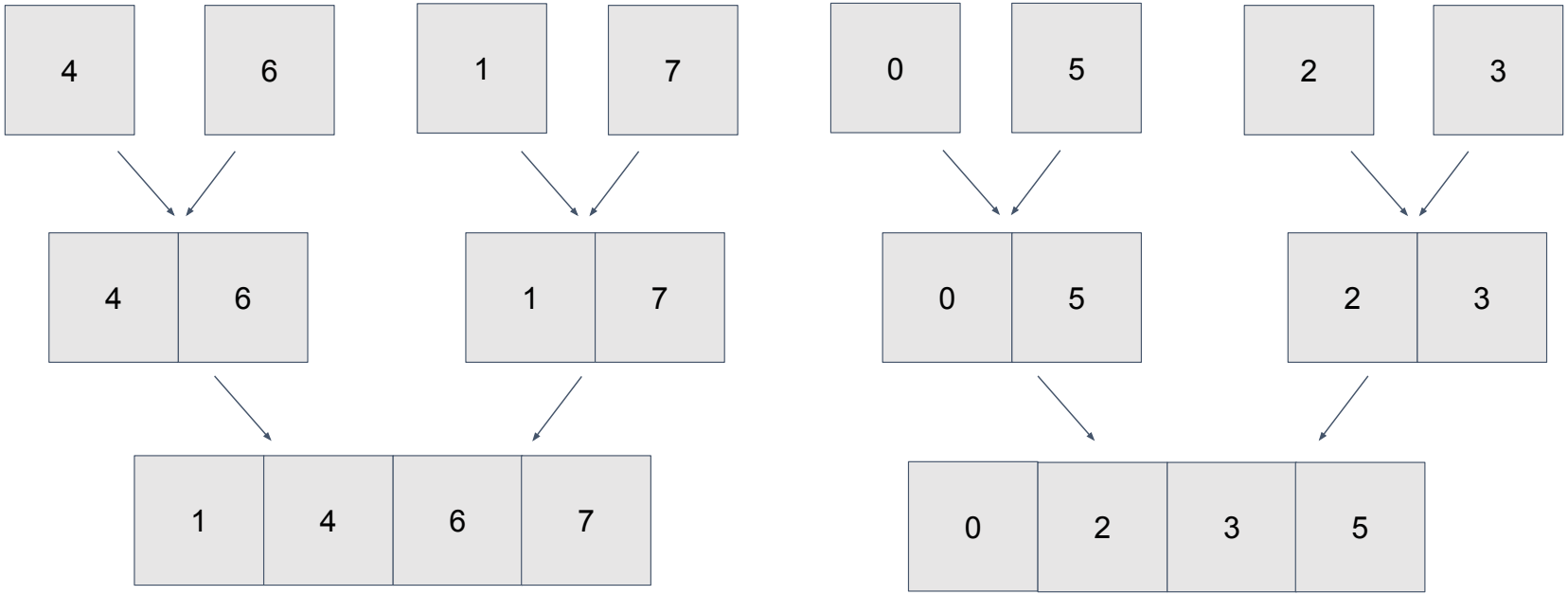


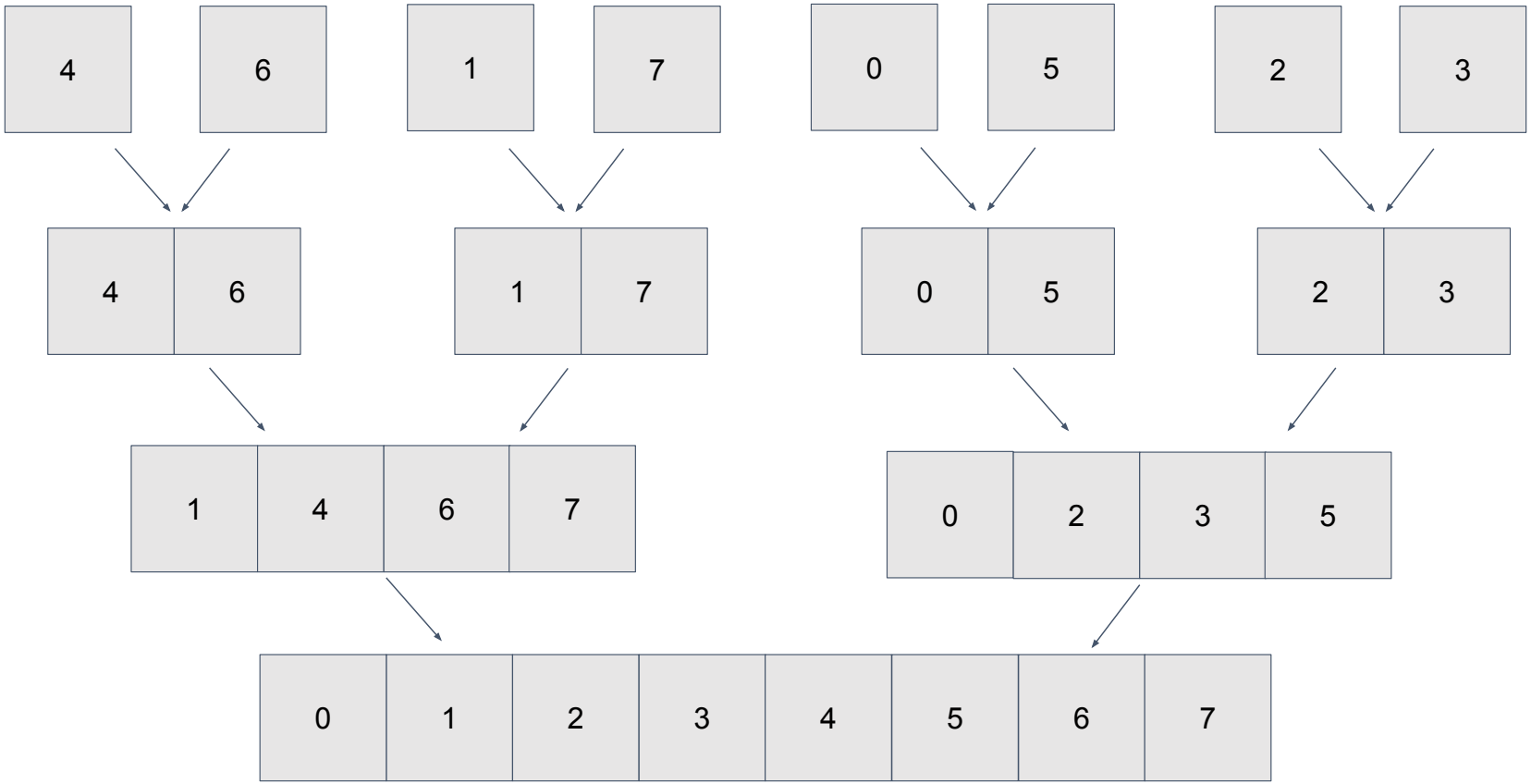
Demonstração - MergeSort

2* Passo - Fazer o merge ordenando



Demonstração - MergeSort




4
6
1
7
0
5
2
3
4
6
1
7
0
5
2
3
1
4
6
7
0
2
3
5
0
1
2
3
4
5
6
7

Demonstração - MergeSort

Vetor ordenado utilizando o MergeSort

0	1	2	3	4	5	6	7

Complexidade - MergeSort

- Melhor caso: ***O(n log n)***;
- Caso médio: ***O(n log n);***
- Pior caso: ***O(n log n)***.

TimSort

- O TimSort é um algoritmo de ordenação híbrido, estável e de alto desempenho;
- Criado por Tim Peters em 2002;
- Utilizado amplamente em Python, Android, Java e outros.

Passos do TimSort

Os passos são bem simples:

1. Dividir o vetor em **runs;**
2. Ordenar cada **run** utilizando o Insertion Sort;
3. Fazer um merge em cada dupla de **runs**.

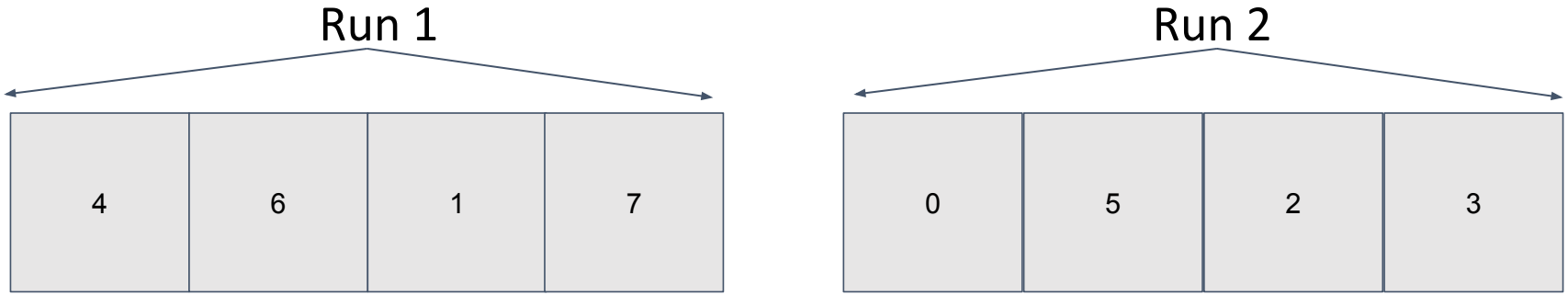
Demonstração - TimSort

Vetor de exemplo utilizado:

4	6	1	7	0	5	2	3

Demonstração - TimSort

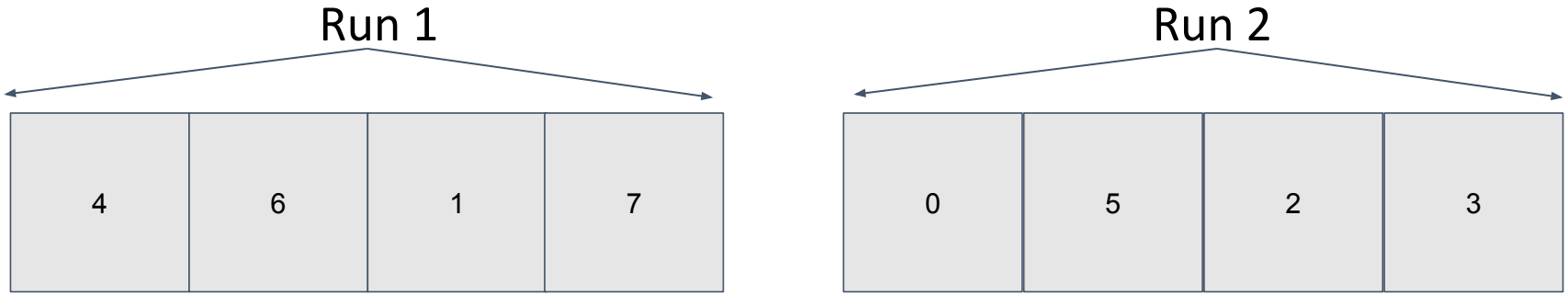
1* Passo - Dividindo o vetor em **runs**



Tamanho da Run = 4

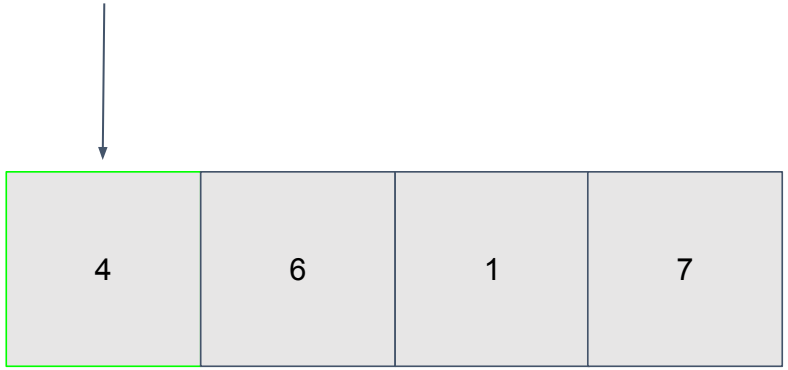
Demonstração - TimSort

2* Passo - Ordenar cada **run** usando o Insertion Sort



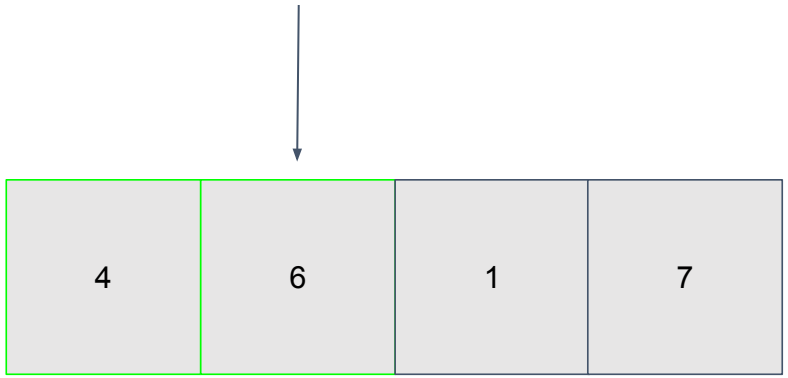
Demonstração - TimSort

Run 1



Demonstração - TimSort

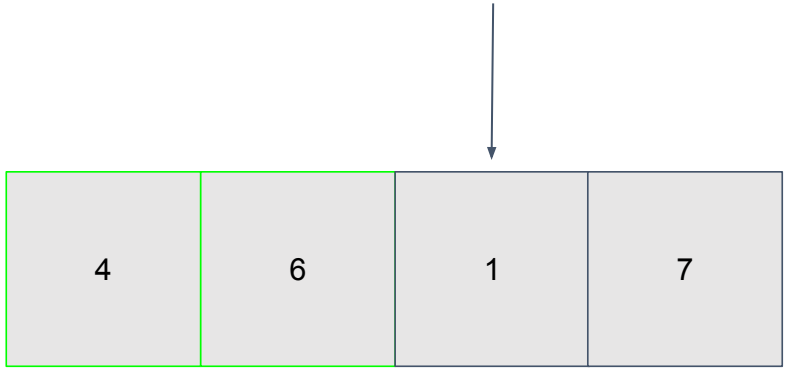
Run 1



6 < 4 SEM TROCA

Demonstração - TimSort

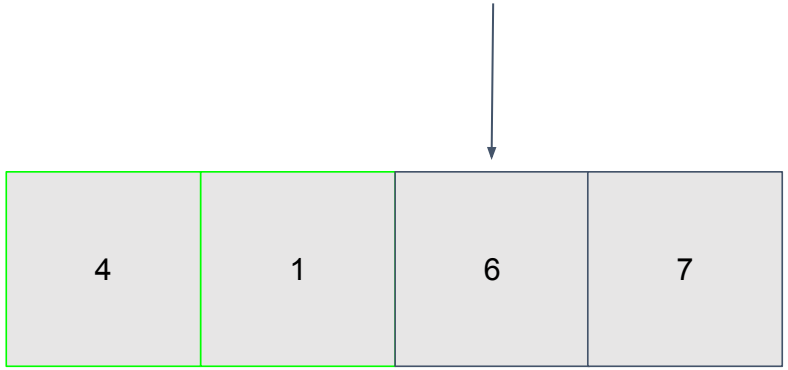
Run 1



1 < 6 TROCA

Demonstração - TimSort

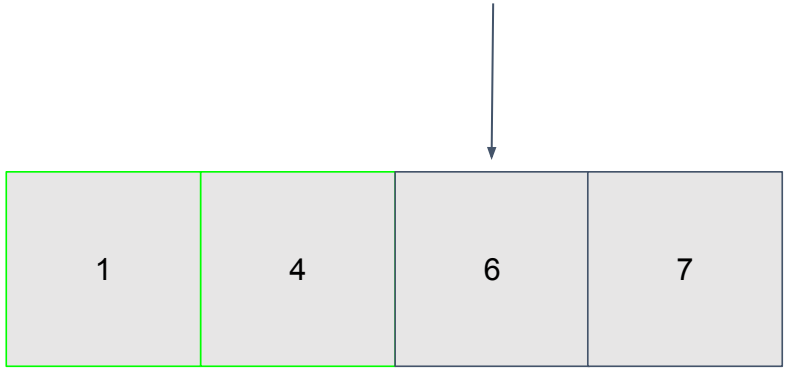
Run 1



1 < 4 TROCA

Demonstração - TimSort

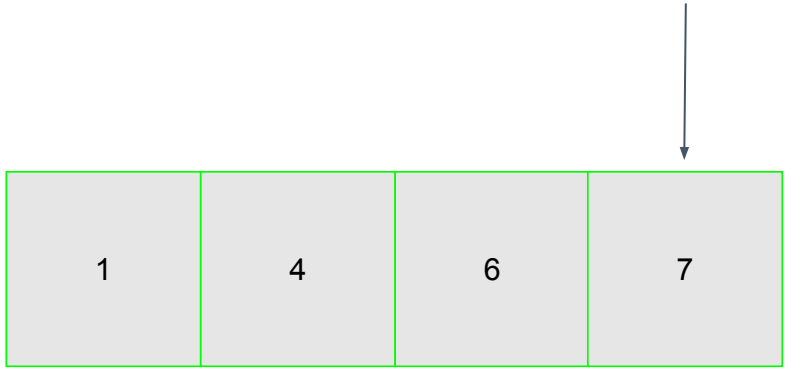
Run 1



1 < 4 TROCA

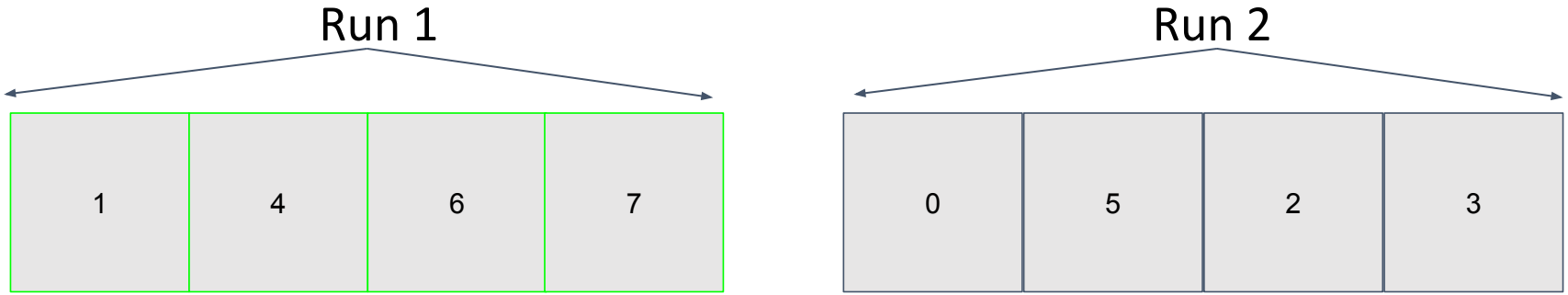
Demonstração - TimSort

Run 1



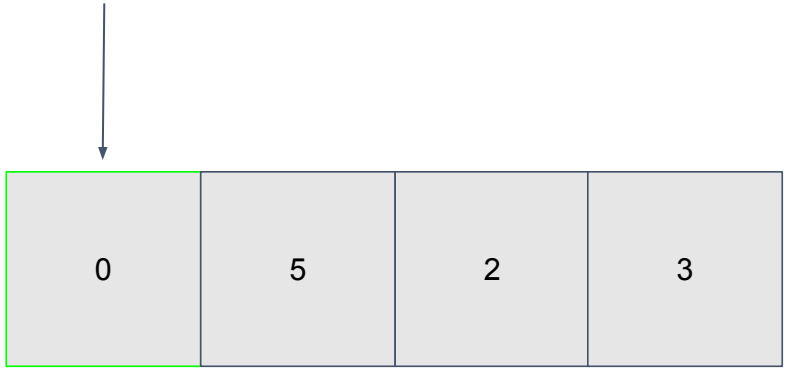
7 < 6 SEM TROCA

Demonstração - TimSort



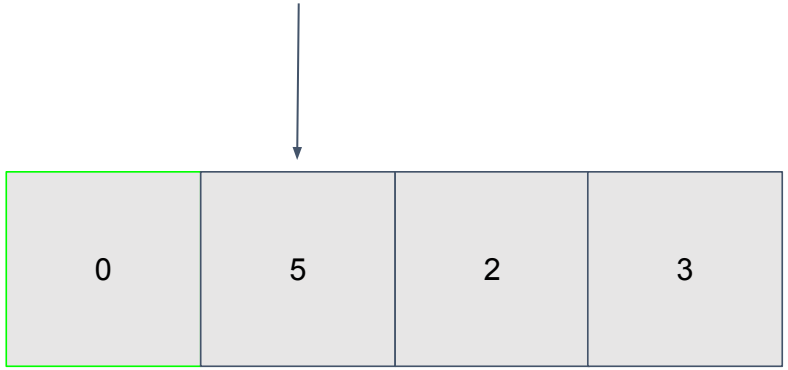
Demonstração - TimSort

Run 2



Demonstração - TimSort

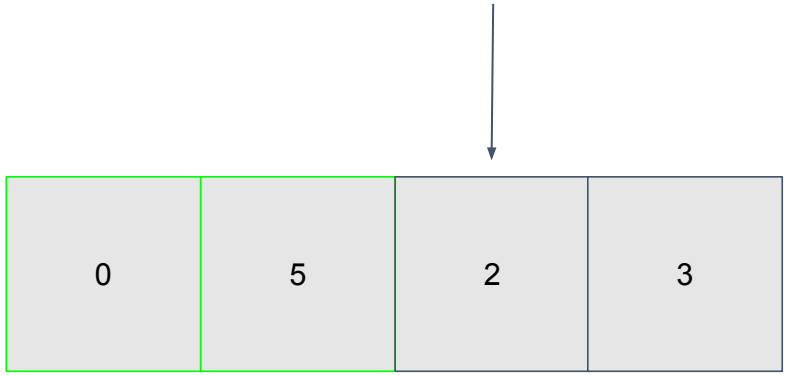
Run 2



5 < 0 SEM TROCA

Demonstração - TimSort

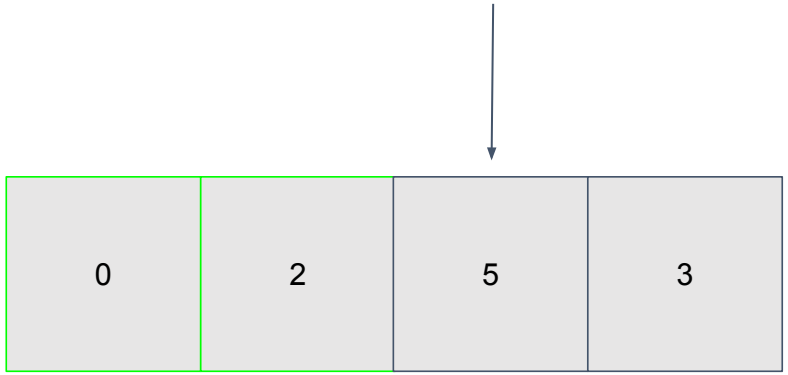
Run 2



2 < 5 TROCA

Demonstração - TimSort

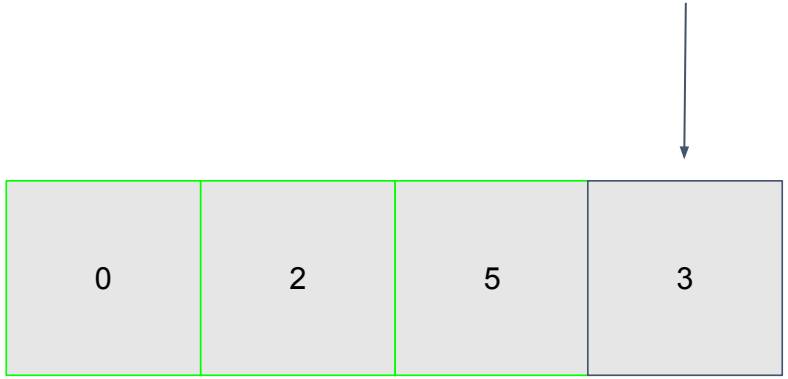
Run 2



2 < 5 TROCA

Demonstração - TimSort

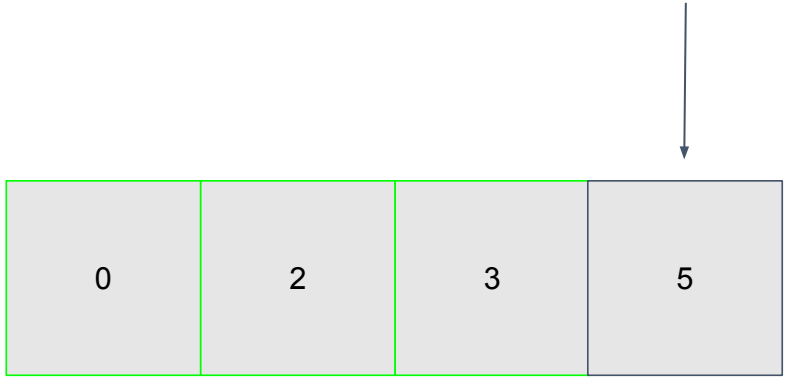
Run 2



3 < 5 TROCA

Demonstração - TimSort

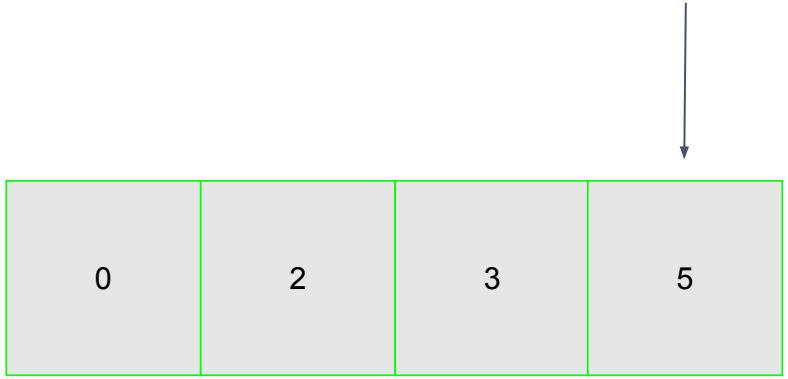
Run 2



3 < 5 TROCA

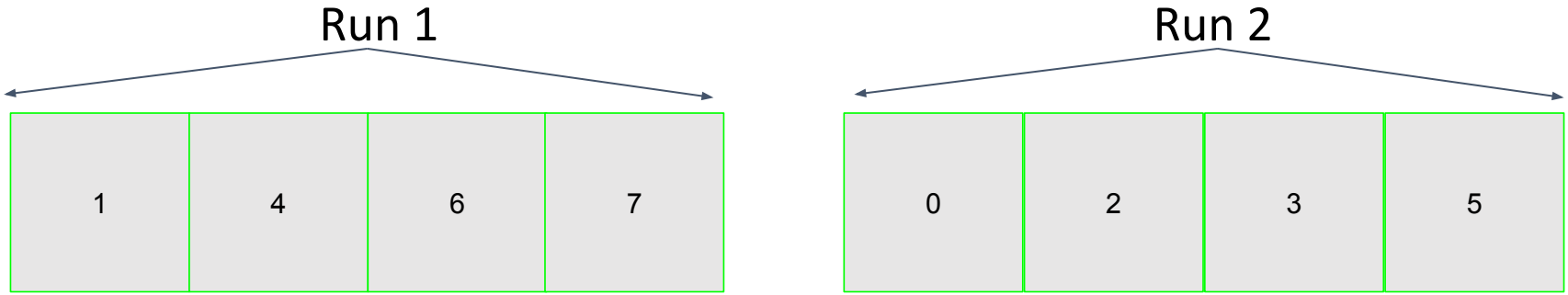
Demonstração - TimSort

Run 2



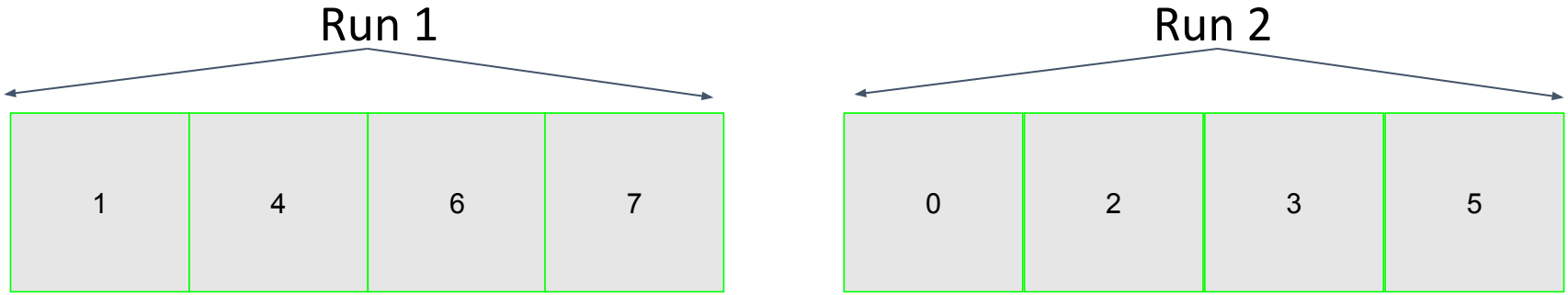
Demonstração - TimSort

Cada **run** foi ordenada usando o insertion sort

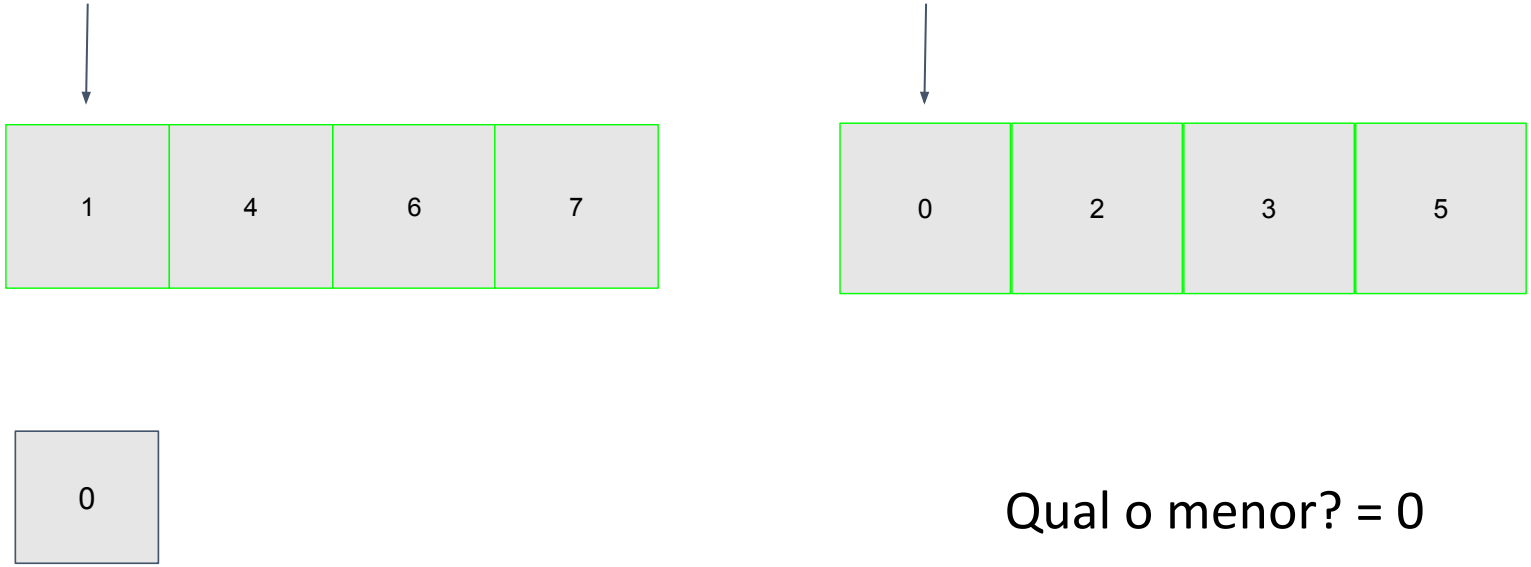


Demonstração - TimSort

3* Passo - Fazer o **merge** na dupla de **runs**

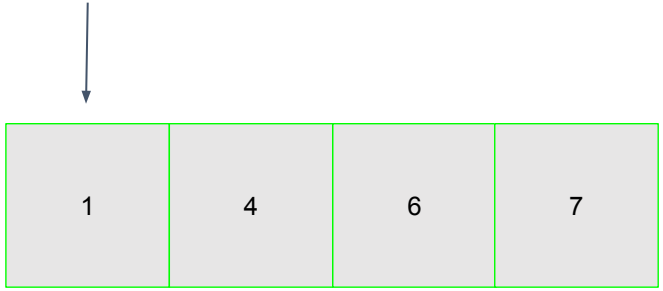


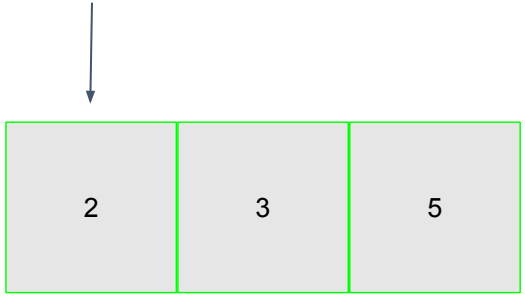
Demonstração - TimSort



Qual o menor? = 0

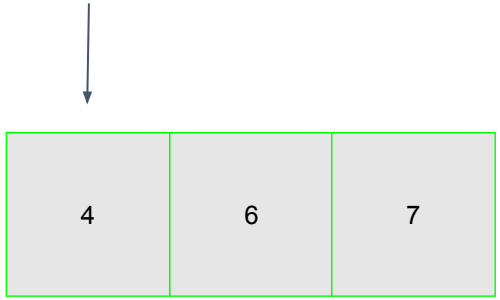
Demonstração - TimSort

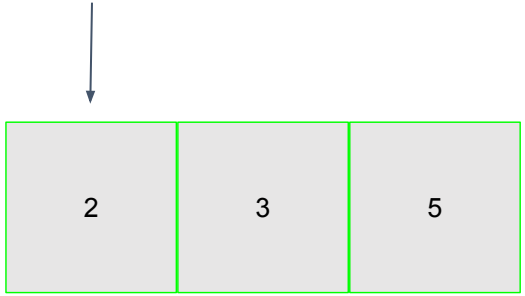




Qual o menor? = 1

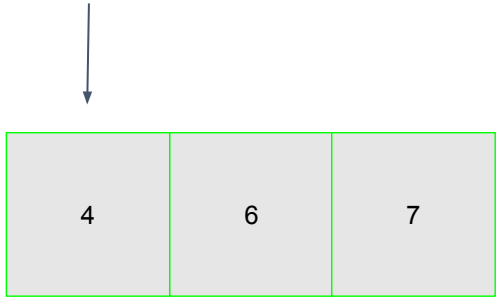
Demonstração - TimSort

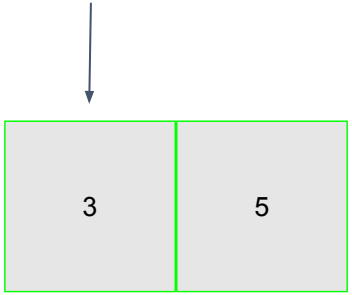


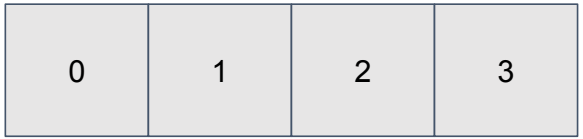


Qual o menor? = 2

Demonstração - TimSort

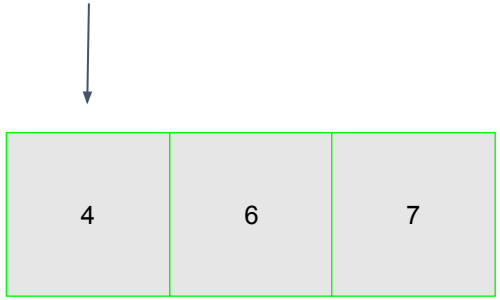


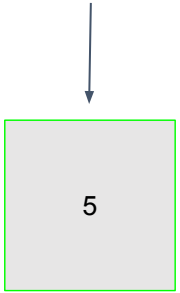




Qual o menor? = 3

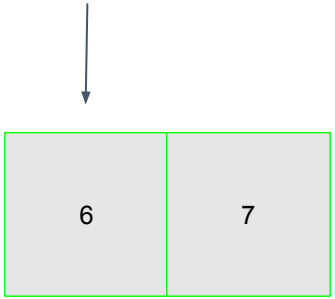
Demonstração - TimSort

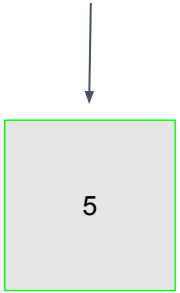




Qual o menor? = 4

Demonstração - TimSort

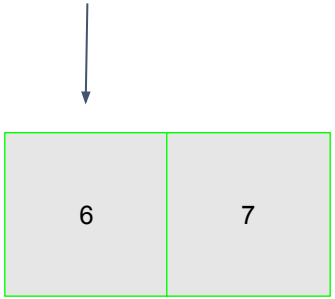




0	1	2	3	4	5

Qual o menor? = 5

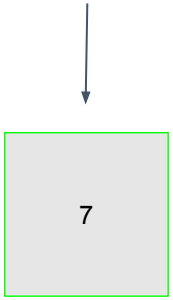
Demonstração - TimSort



0	1	2	3	4	5	6

Qual o menor? = 6

Demonstração - TimSort



0	1	2	3	4	5	6	7

Qual o menor? = 7

Demonstração - TimSort

Vetor ordenado utilizando o TimSort

0	1	2	3	4	5	6	7

Complexidade - TimSort

- Melhor caso: ***O(n)***;
- Caso médio: ***O(n log n);***
- Pior caso: ***O(n log n)***.

Resultados e Discussão

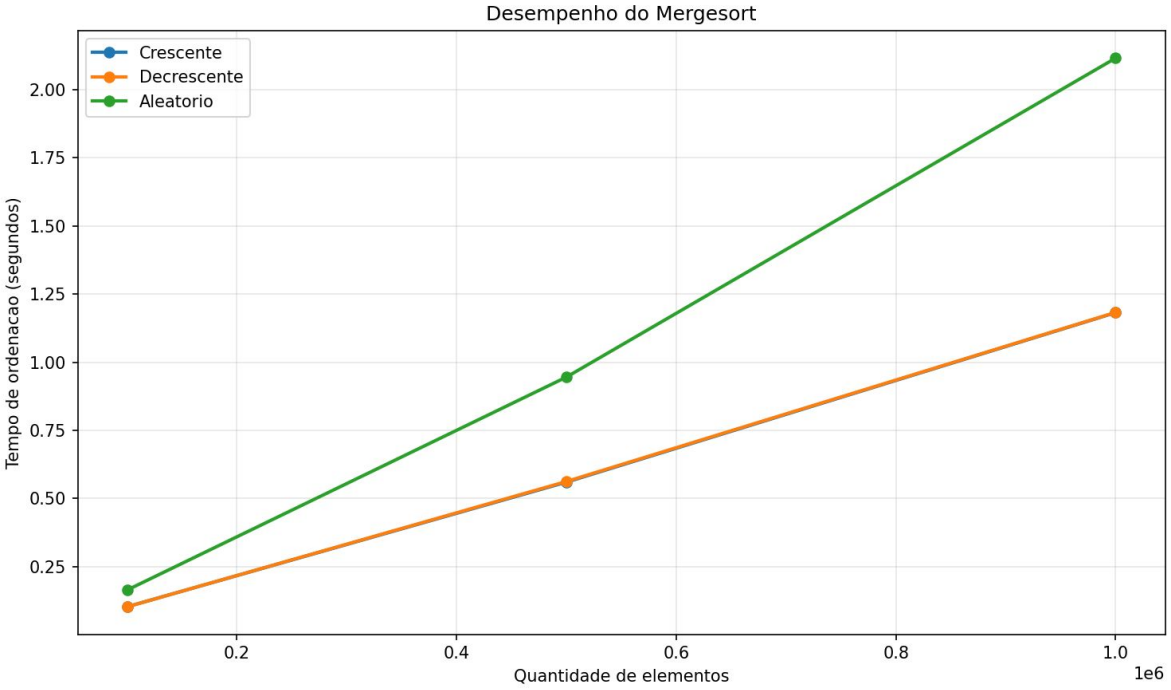


Gráfico 1: Tempo Médio de Execução - Merge Sort

Fonte: Elaboração Própria, 2026.

Resultados e Discussão

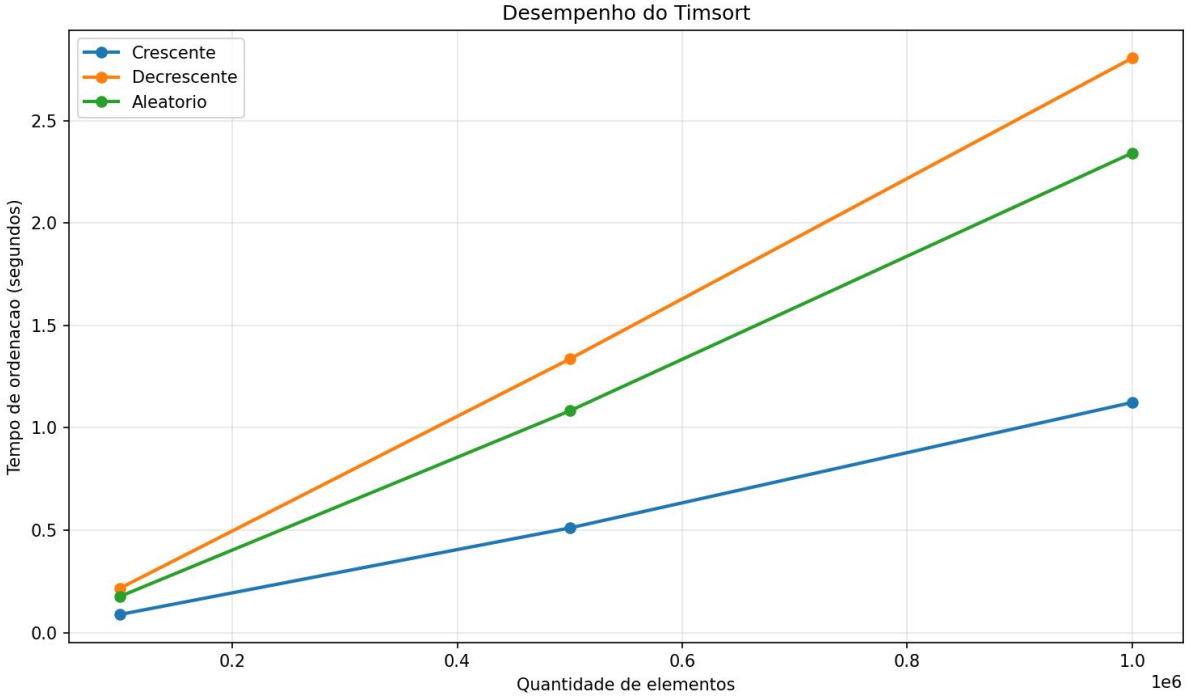


Gráfico 2: Tempo Médio de Execução - Tim Sort.

Fonte: Elaboração Própria, 2026.

Resultados e Discussão

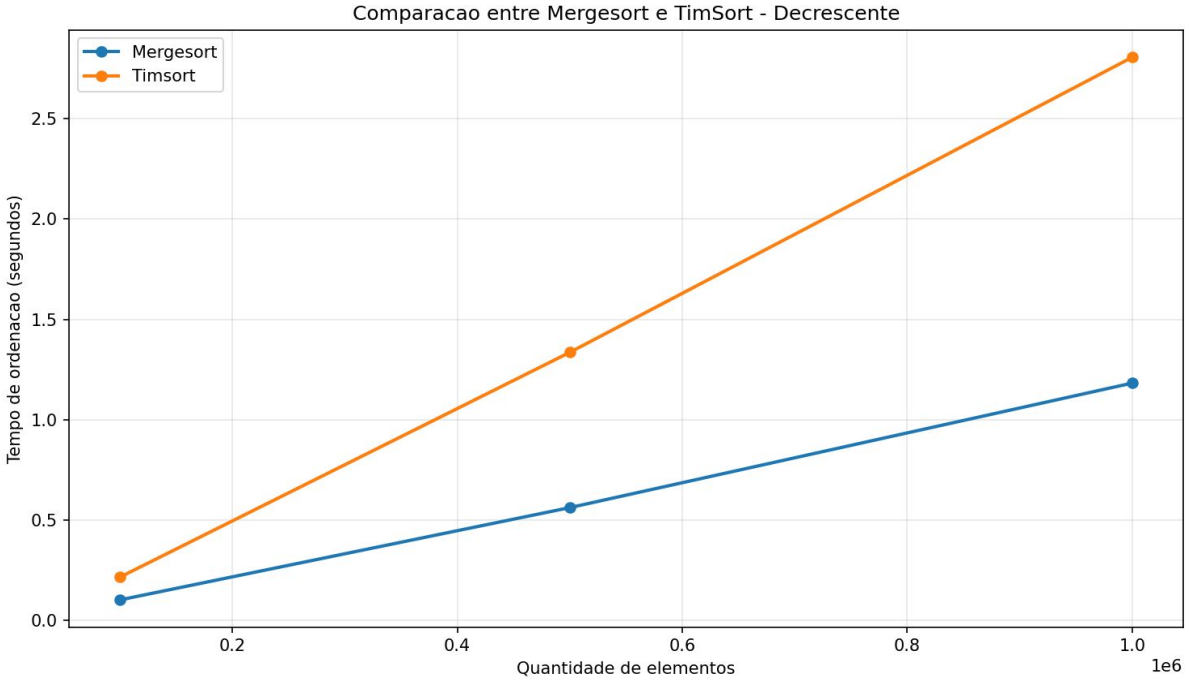


Gráfico 3: Comparativo em caso decrescente

Fonte: Elaboração Própria, 2026.

Resultados e Discussão

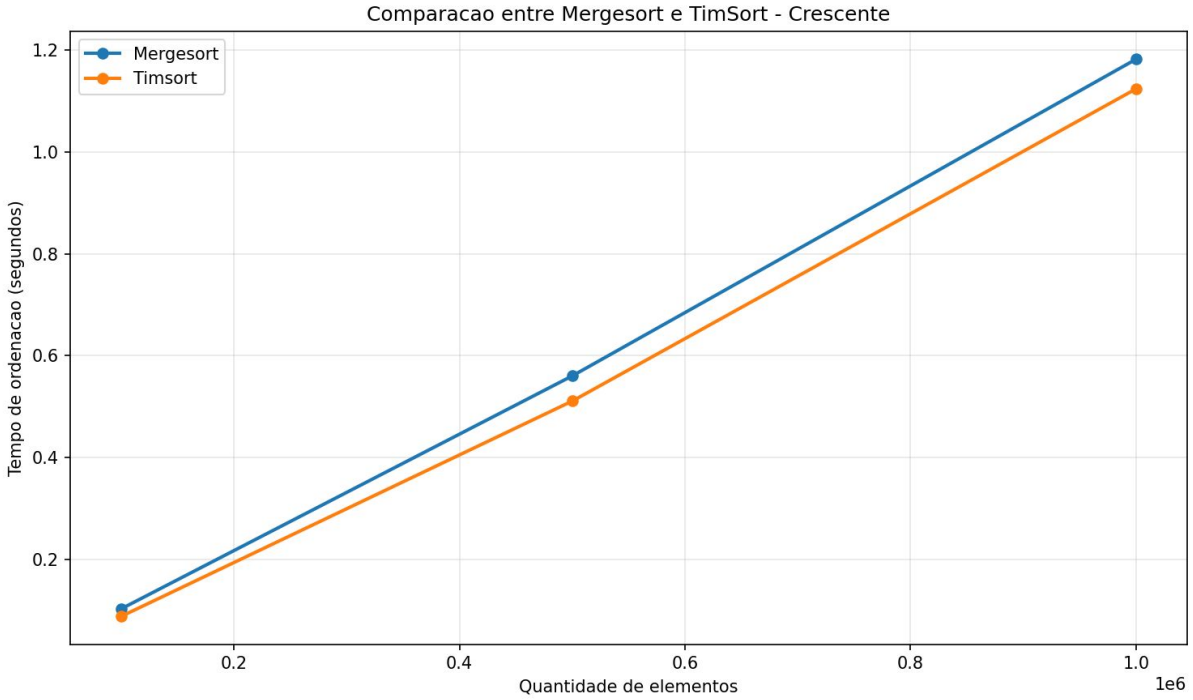


Gráfico 4: Comparativo em caso crescente

Fonte: Elaboração Própria, 2026.

Resultados e Discussão

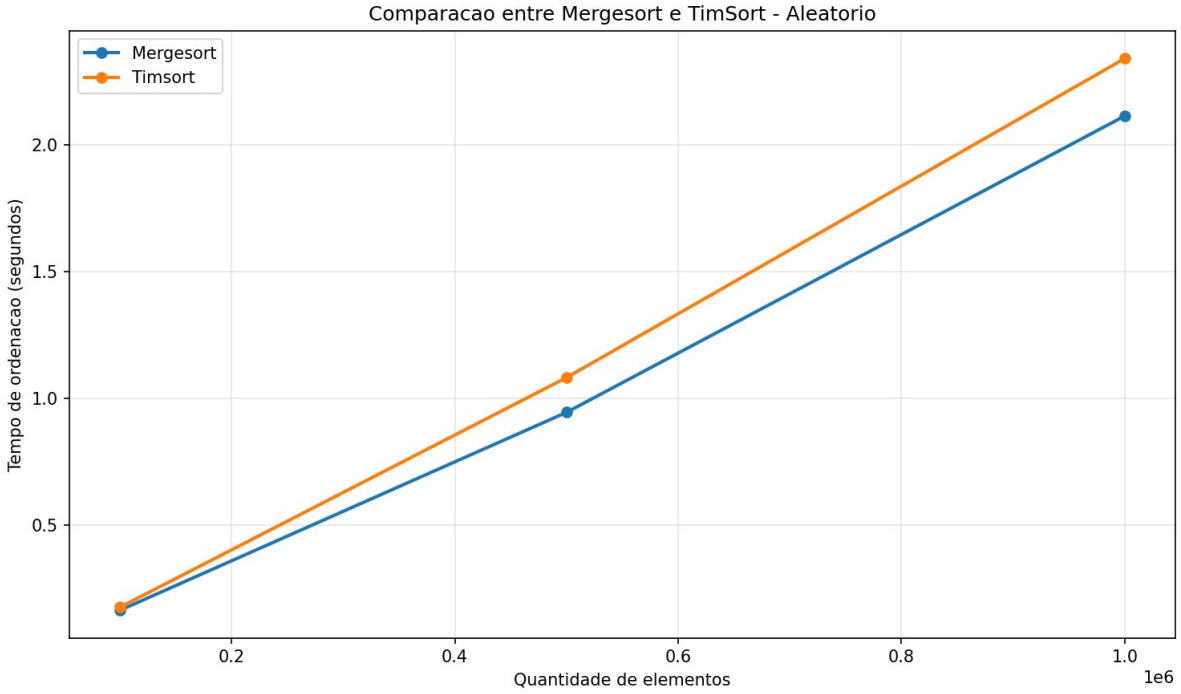


Gráfico 5: Comparativo em caso aleatório.

Fonte: Elaboração Própria, 2026.

Conclusão

- MergeSort é eficiente e consistente, com desempenho estável independentemente da ordem dos dados;
- TimSort é adaptativo e pode ser mais rápido em dados já ordenados, mas perde desempenho em cenários desordenados;
- Não existe o algoritmo melhor em todos os casos, mas sim o mais adequado a cada caso.

Referências

Aho, A.V., Hopcroft, J. E., and Ullman, J. D. (1974). The Design and Analysis of Computer Algorithms. Addison-Wesley.

Auger, N., Jugé, V., Nicaud, C., and Pivoteau, C. (2018). On the worst-case complexity of TimSort. In 26th Annual European Symposium on Algorithms (ESA 2018), volume 112 of Leibniz International Proceedings in Informatics (LIPIcs), pages 4:1–4:13. Schloss Dagstuhl– Leibniz-Zentrum für Informatik.

Bentley, J. L. and McIlroy, M. D. (1993). Engineering a sort function. Software: Practice and Experience, 23(11):1249–1265.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). Introduction to Algorithms. MIT Press, Cambridge, MA, 3 edition.

Referências

Gonnet, G. H. and Baeza-Yates, R. (1991). Handbook of Algorithms and Data Structures. Addison-Wesley.

Knuth, D. E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching. Addison-Wesley, Reading, MA, 2 edition.

McIlroy, P. M. (1993). Optimistic sorting and information theoretic complexity. pages 467–474.

Peters, T. (2002). TimSort description. CPython source repository.

Sedgewick, R. and Wayne, K. (2011). Algorithms. Addison-Wesley Professional, Upper Saddle River, NJ, 4 edition.

Weiss, M. A. (2014). Data Structures and Algorithm Analysis in Java. Pearson, 3 edition.

Referências

Wiredu, J. K., Akobre, S., Aabaah, I., and Wumpini, U. A. (2025). A unified framework for theoretical and experimental evaluation of classical and modern sorting algorithms in real-time systems. Journal of Information Systems and Informatics, 7(4):3143–3185.